```
import re, nltk, spacy, torch
import torch.nn as nn
import numpy as np
from collections import Counter

from nltk import word_tokenize, pos_tag, CFG, PCFG, ChartParser,
ViterbiParser
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.lm import MLE, Laplace, Lidstone
from nltk.lm.preprocessing import padded_everygram_pipeline
from nltk.corpus import wordnet
from nltk.translate.bleu_score import sentence_bleu

from sklearn.feature_extraction.text import CountVectorizer,
TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity,
euclidean_distances
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score
from gensim.models import Word2Vec
from transformers import pipeline, AutoTokenizer, AutoModel

nltk.download(["punkt", "punkt_tab", "averaged_perceptron_tagger_eng",
"wordnet", "omw-1.4"], quiet=True)
nlp = spacy.load("en_core_web_sm")

sentences = [
    "I like natural language processing.",
    "I like machine learning and deep learning.",
    "The boy saw the man with the telescope.",
    "Mr. Rahul bought a laptop in Chennai. He loved it."
]

text = "I'm learning NLP at http://test.com!! Running models is fun."
text = re.sub(r"http\S+|[^a-zA-Z\s]", "", text.lower())
tokens = word_tokenize(text)

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

print("Tokens:", tokens)
print("Stems :", [stemmer.stem(w) for w in tokens])
print("Lemmas:", [lemmatizer.lemmatize(w, pos="v") for w in tokens])

Tokens: ['im', 'learning', 'nlp', 'at', 'running', 'models', 'is',
'fun']
Stems : ['im', 'learn', 'nlp', 'at', 'run', 'model', 'is', 'fun']
Lemmas: ['im', 'learn', 'nlp', 'at', 'run', 'model', 'be', 'fun']
```

```
tokenized = [word_tokenize(s.lower()) for s in sentences]

# 1. Unsmoothed Bigram (MLE)
train_mle, vocab = padded_everygram_pipeline(2, tokenized)
mle = MLE(2)
mle.fit(train_mle, vocab)

# 2. Add-One / Laplace Smoothing
train_lap, vocab = padded_everygram_pipeline(2, tokenized)
laplace = Laplace(2)
laplace.fit(train_lap, vocab)

# 3. Add-k / Lidstone Smoothing (gamma = 0.5)
train_lid, vocab = padded_everygram_pipeline(2, tokenized)
lidstone = Lidstone(0.5, 2)
lidstone.fit(train_lid, vocab)

print("MLE P(like | i)     :", mle.score("like", ["i"]))
print("Laplace P(like | i) :", round(laplace.score("like", ["i"]), 4))
print("Lidstone P(like | i):", round(lidstone.score("like", ["i"]),
4))
```

```
MLE P(like | i)     : 1.0
Laplace P(like | i) : 0.0968
Lidstone P(like | i): 0.1515
```

```
words = "i like natural language processing".split()
w2i = {w: i for i, w in enumerate(words)}

X = torch.tensor([w2i[w] for w in words[:-1]])  # Current word
y = torch.tensor([w2i[w] for w in words[1:]])   # Next word

model = nn.Sequential(nn.Embedding(len(words), 8), nn.Linear(8,
len(words)))
opt = torch.optim.Adam(model.parameters(), lr=0.05)
loss_fn = nn.CrossEntropyLoss()

for _ in range(50):
    opt.zero_grad()
    loss = loss_fn(model(X), y)
    loss.backward()
    opt.step()

pred_idx = model(X).argmax(dim=-1).tolist()
print("Predicted Next Words:", [words[i] for i in pred_idx])
```

```
Predicted Next Words: ['like', 'natural', 'language', 'processing']
```

```
tokens = word_tokenize(sentences[0])
print("POS Tags:", pos_tag(tokens))
```

```
POS Tags: [('I', 'PRP'), ('like', 'VBP'), ('natural', 'JJ'),
('language', 'NN'), ('processing', 'NN'), ('.', '.')]
```

```
cfg = CFG.fromstring("""
S -> NP VP
NP -> Det N | Det N PP
VP -> V NP | VP PP
PP -> P NP
Det -> 'the'
N -> 'boy' | 'man' | 'telescope'
V -> 'saw'
P -> 'with'
""")

parser = ChartParser(cfg)
sent = "the boy saw the man with the telescope".split()
for tree in parser.parse(sent):
    tree.pretty_print()
```

```
                  S
      ____________|_______
     |                    VP
     |                ____|________
     |               VP             PP
     |           ____|___          ____|___
     NP         |        NP       |        NP
  ___|___      |     ___|___      |     ___|______
Det      N     V   Det      N     P   Det         N
 |       |     |    |       |     |    |          |
the     boy   saw  the     man   with the     telescope
```

```
                  S
      ____________|___
     |                VP
     |          ______|___
     |         |          NP
     |         |      ____|____
     |         |     |         PP
     |         |     |     ____|___
     NP        |     |    |        NP
  ___|___      |     |    |     ___|______
Det      N     V   Det   N     P   Det         N
 |       |     |    |    |     |    |          |
the     boy   saw  the  man   with the     telescope
```

```
pcfg = PCFG.fromstring("""
S -> NP VP [1.0]
NP -> Det N [0.6] | Det N PP [0.4]
VP -> V NP [0.7] | VP PP [0.3]
```

```
PP -> P NP [1.0]
Det -> 'the' [1.0]
N -> 'boy' [0.4] | 'man' [0.3] | 'telescope' [0.3]
V -> 'saw' [1.0]
P -> 'with' [1.0]
""")

viterbi = ViterbiParser(pcfg)
for tree in viterbi.parse(sent):
    print("Most Probable PCFG Tree:")
    tree.pretty_print()
```

```
Most Probable PCFG Tree:
                  S
      ____________|___
     |                VP
     |            ____|___
     |           |        NP
     |           |     ____|____
     |           |    |         PP
     |           |    |     ____|___
     NP          |    |    |        NP
  ___|___        |    |    |     ___|______
Det      N      V  Det   N    P   Det      N
 |       |      |   |    |    |    |       |
the     boy    saw the  man with the  telescope
```

```
doc = nlp(sentences[0])
for token in doc:
    print(f"{token.text:12} --({token.dep_:8})--> {token.head.text}")
```

```
I            --(nsubj   )--> like
like         --(ROOT    )--> like
natural      --(amod    )--> language
language     --(compound)--> processing
processing   --(dobj    )--> like
.            --(punct   )--> like
```

```
##UNIT III: WordNet, Vector Space Models, Similarity, Embeddings &
Classification

s1 = wordnet.synsets("machine")[0]
s2 = wordnet.synsets("computer")[0]

print("Synset:", s1.name(), "| Meaning:", s1.definition())
print("Wu-Palmer Similarity:", round(s1.wup_similarity(s2), 3))
```

```
Synset: machine.n.01 | Meaning: any mechanical or electrical device
that transmits or modifies energy to perform or assist in the
```

```
performance of human tasks
Wu-Palmer Similarity: 0.941
```

```
bow = CountVectorizer().fit_transform(sentences).toarray()
tfidf = TfidfVectorizer().fit_transform(sentences).toarray()

print("BoW Shape:", bow.shape, "| TF-IDF Shape:", tfidf.shape)
print("TF-IDF First Row:", tfidf[0].round(2))
```

```
BoW Shape: (4, 23) | TF-IDF Shape: (4, 23)
TF-IDF First Row: [0.   0.   0.   0.   0.   0.   0.   0.   0.53 0.
0.   0.41 0.   0.
 0.   0.   0.53 0.53 0.   0.   0.   0.   0.  ]
```

```
cos_sim = cosine_similarity([tfidf[0]], [tfidf[1]])[0][0]
euc_dist = euclidean_distances([tfidf[0]], [tfidf[1]])[0][0]

print("Cosine Similarity :", round(cos_sim, 3))
print("Euclidean Distance:", round(euc_dist, 3))
```

```
Cosine Similarity : 0.118
Euclidean Distance: 1.328
```

```
w2v = Word2Vec(tokenized, vector_size=20, window=2, min_count=1, sg=1,
seed=42)
print("Vector for 'learning' (first 5):", w2v.wv["learning"]
[:5].round(3))
print("Similar to 'learning':", w2v.wv.most_similar("learning",
topn=2))
```

```
Vector for 'learning' (first 5): [-0.033  0.026  0.02  -0.015 -0.043]
Similar to 'learning': [('loved', 0.33389076590538025), ('boy',
0.2926388084888458)]
```

```
train_x = ["I love this movie", "Great film and acting", "Terrible
movie", "Worst acting ever"]
train_y = [1, 1, 0, 0]

vec = TfidfVectorizer()
X_vec = vec.fit_transform(train_x)

clf = MultinomialNB().fit(X_vec, train_y)
pred = clf.predict(vec.transform(["Great movie"]))
print("Text Classification Prediction:", "Positive" if pred[0] == 1
else "Negative")
```

```
Text Classification Prediction: Positive
```

##UNIT IV: Coreference, RNN Models, Machine Translation & Evaluation Metrics

```
doc = nlp("Mr. Rahul bought a laptop. He loved it.")
persons = [e.text for e in doc.ents if e.label_ == "PERSON"]
nouns = [c.text for c in doc.noun_chunks if c.text not in persons and
c.root.pos_ != "PRON"]

for token in doc:
    if token.text.lower() in ["he", "she", "him", "her"] and persons:
        print(f"Coreference: '{token.text}' refers to '{persons[-
1]}'")
    elif token.text.lower() == "it" and nouns:
        print(f"Coreference: '{token.text}' refers to '{nouns[-1]}'")
```

```
Coreference: 'He' refers to 'Rahul'
Coreference: 'it' refers to 'a laptop'
```

```
rnn = nn.RNN(input_size=8, hidden_size=16, batch_first=True)
fc = nn.Linear(16, 2)

sample_input = torch.randn(1, 5, 8)  # (batch=1, seq_len=5,
features=8)
output, hidden = rnn(sample_input)
logits = fc(hidden[-1])
print("RNN Output Logits:", logits.detach().numpy().round(3))
```

```
RNN Output Logits: [[0.273 0.108]]
```

```
translator = pipeline("translation_en_to_fr",
model="Helsinki-NLP/opus-mt-en-fr")
print("Translation:", translator("I like natural language
processing.")[0]["translation_text"])
```

```
C:\Users\rohith\AppData\Local\Programs\Python\Python311\Lib\site-
packages\huggingface_hub\file_download.py:949: FutureWarning:
`resume_download` is deprecated and will be removed in version 1.0.0.
Downloads always resume when possible. If you want to force a new
download, use `force_download=True`.
  warnings.warn(
```


```
{"model_id":"f1f03cdcfba6415b9a51154bb4be8434","version_major":2,"vers
ion_minor":0}

{"model_id":"e909968508f1477193ae03fe5535e73c","version_major":2,"vers
ion_minor":0}

{"model_id":"92a52b434e1c422c9d6c402d0fe79941","version_major":2,"vers
ion_minor":0}

{"model_id":"53f0390805a74ed0940234eaf6eca024","version_major":2,"vers
ion_minor":0}

{"model_id":"28c534f1d10b49579311349c82e63633","version_major":2,"vers
ion_minor":0}
```

```
{"model_id":"fd8bad569f1a4155b6ae6e564cec9dcf","version_major":2,"vers
ion_minor":0}

{"model_id":"0f1a24384d924a648bf4124d9cf0b2e9","version_major":2,"vers
ion_minor":0}

C:\Users\rohith\AppData\Local\Programs\Python\Python311\Lib\site-
packages\transformers\models\marian\tokenization_marian.py:175:
UserWarning: Recommended: pip install sacremoses.
  warnings.warn("Recommended: pip install sacremoses.")

Translation: J'aime le traitement du langage naturel.
```

```
# 1. BLEU Score for Machine Translation / Generation
ref = [["the", "cat", "sat", "on", "the", "mat"]]
cand = ["the", "cat", "is", "on", "the", "mat"]
print("BLEU Score:", round(sentence_bleu(ref, cand), 4))

# 2. Classification Metrics
y_true, y_pred = [1, 0, 1, 1], [1, 0, 0, 1]
print("Accuracy :", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall   :", recall_score(y_true, y_pred))
print("F1 Score :", round(f1_score(y_true, y_pred), 3))
```

```
BLEU Score: 0.0
Accuracy : 0.75
Precision: 1.0
Recall   : 0.6666666666666666
F1 Score : 0.8

C:\Users\rohith\AppData\Local\Programs\Python\Python311\Lib\site-
packages\nltk\translate\bleu_score.py:577: UserWarning:
The hypothesis contains 0 counts of 4-gram overlaps.
Therefore the BLEU score evaluates to 0, independently of
how many N-gram overlaps of lower order it contains.
Consider using lower n-gram order or use SmoothingFunction()
  warnings.warn(_msg)
```

```
##UNIT V: Attention Mechanisms, Transformers, BERT & NER

Q = torch.randn(3, 4)
K = torch.randn(3, 4)
V = torch.randn(3, 4)

scores = (Q @ K.T) / np.sqrt(4)
weights = torch.softmax(scores, dim=-1)
attention_out = weights @ V

print("Attention Weights:\n", weights.numpy().round(3))
print("Attention Output:\n", attention_out.numpy().round(3))
```

```
Attention Weights:
 [[0.336 0.362 0.302]
 [0.396 0.372 0.233]
 [0.298 0.206 0.496]]
Attention Output:
 [[ 0.525  0.382  0.088 -0.733]
 [ 0.404  0.346  0.054 -0.894]
 [ 0.792  0.204 -0.055 -0.357]]
```

```
sentiment = pipeline("sentiment-analysis")
print("Transformer Sentiment:", sentiment("This lab exam is very
easy!"))
```

```
No model was supplied, defaulted to distilbert/distilbert-base-
uncased-finetuned-sst-2-english and revision af0f99b
(https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-
sst-2-english).
Using a pipeline without specifying a model name and revision in
production is not recommended.
C:\Users\rohith\AppData\Local\Programs\Python\Python311\Lib\site-
packages\huggingface_hub\file_download.py:949: FutureWarning:
`resume_download` is deprecated and will be removed in version 1.0.0.
Downloads always resume when possible. If you want to force a new
download, use `force_download=True`.
  warnings.warn(

Transformer Sentiment: [{'label': 'POSITIVE', 'score':
0.9849307537078857}]
```

```
# 1. BERT Fill-Mask
fill = pipeline("fill-mask", model="bert-base-uncased")
print("BERT Fill-Mask:", fill("Natural language [MASK] is fun.")[0]
["sequence"])

# 2. BERT Contextual Embeddings
tok = AutoTokenizer.from_pretrained("bert-base-uncased")
bert = AutoModel.from_pretrained("bert-base-uncased")
inputs = tok("I like NLP", return_tensors="pt")
emb = bert(**inputs).last_hidden_state
print("BERT Embedding Shape:", emb.shape)
```

```
Some weights of the model checkpoint at bert-base-uncased were not
used when initializing BertForMaskedLM: ['bert.pooler.dense.bias',
'bert.pooler.dense.weight', 'cls.seq_relationship.bias',
'cls.seq_relationship.weight']
- This IS expected if you are initializing BertForMaskedLM from the
checkpoint of a model trained on another task or with another
architecture (e.g. initializing a BertForSequenceClassification model
from a BertForPreTraining model).
- This IS NOT expected if you are initializing BertForMaskedLM from
```

```
the checkpoint of a model that you expect to be exactly identical
(initializing a BertForSequenceClassification model from a
BertForSequenceClassification model).

BERT Fill-Mask: natural language learning is fun.
BERT Embedding Shape: torch.Size([1, 6, 768])
```

```
doc = nlp(sentences[3])  # "Mr. Rahul bought a laptop in Chennai. He
loved it."
for ent in doc.ents:
    print(f"Entity: {ent.text:12} | Label: {ent.label_}")
```

```
Entity: Rahul        | Label: PERSON
Entity: Chennai      | Label: GPE
```